

## Table of Contents

|                                                                           |    |
|---------------------------------------------------------------------------|----|
| WhatsApp Integration Docs.....                                            | 4  |
| Root Directory (whatsapp-int-repo) .....                                  | 4  |
| agent Folder .....                                                        | 4  |
| microservice Folder .....                                                 | 4  |
| webhook_fa Folder .....                                                   | 4  |
| website Folder .....                                                      | 5  |
| Now let's explain a little bit more the docker compose file Services..... | 5  |
| Volumes .....                                                             | 7  |
| Networks .....                                                            | 7  |
| Nginx Configuration Overview .....                                        | 8  |
| Docker Compose File .....                                                 | 8  |
| How URL Routing Works .....                                               | 9  |
| Flask App Documentation .....                                             | 10 |
| Blueprints .....                                                          | 10 |
| Auth Blueprint.....                                                       | 10 |
| Campaigns Blueprint.....                                                  | 10 |
| Templates Blueprint .....                                                 | 11 |
| Upload Data Blueprint .....                                               | 12 |
| FastAPI Application Documentation.....                                    | 12 |
| Overview .....                                                            | 12 |
| Project Structure.....                                                    | 12 |
| API Endpoints .....                                                       | 13 |
| Webhook Endpoint .....                                                    | 13 |
| Initialization Scripts.....                                               | 13 |
| MongoDB Initialization (init_mongo.py).....                               | 13 |
| Webhook URL Initialization (init_webhook_urls.py).....                    | 14 |
| Utilities.....                                                            | 14 |
| Auto-Reply Handler (auto_reply_helper.py) .....                           | 14 |
| Dialog 360 Integration (dialog_360.py).....                               | 14 |
| Helper Functions (helper_functions.py).....                               | 14 |
| Environment Variable Management (utils_init.py).....                      | 15 |

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| Foundation Handlers .....                                                     | 15 |
| Initialization ( <code>init_webhook_helper.py</code> ) .....                  | 15 |
| Setup .....                                                                   | 15 |
| Environment Variables .....                                                   | 15 |
| Microservice Documentation .....                                              | 16 |
| Overview .....                                                                | 16 |
| Project Structure.....                                                        | 16 |
| API Endpoints.....                                                            | 16 |
| 1. Start Campaign Task .....                                                  | 16 |
| 2. Get Campaign Task Status .....                                             | 17 |
| Task Workflow .....                                                           | 18 |
| Celery Task: <code>send_message_campaign</code> .....                         | 18 |
| Utilities.....                                                                | 19 |
| Chat Room Helper ( <code>chat_rooms_helper.py</code> ) .....                  | 19 |
| Foundation Session Manager ( <code>manage_foundation_session.py</code> )..... | 19 |
| Campaign Helpers ( <code>send_campaigns_helpers.py</code> ).....              | 19 |
| Environment Variables .....                                                   | 20 |
| Agent Microservice Documentation.....                                         | 20 |
| Overview .....                                                                | 20 |
| Project Structure.....                                                        | 20 |
| Socket.IO Events Documentation.....                                           | 21 |
| Namespace Initialization.....                                                 | 21 |
| Event: <code>connect</code> .....                                             | 21 |
| Event: <code>available_rooms</code> .....                                     | 21 |
| Event: <code>new_room</code> .....                                            | 22 |
| Event: <code>message_from_user</code> .....                                   | 22 |
| Event: <code>message_from_business</code> .....                               | 23 |
| Event: <code>join_room</code> .....                                           | 23 |
| Event: <code>close_conversation</code> .....                                  | 24 |
| Frontend Events.....                                                          | 24 |
| Available Events.....                                                         | 24 |
| Database Schema.....                                                          | 25 |
| Collection: <code>rooms</code> .....                                          | 25 |

Environment Variables .....25

# WhatsApp Integration Docs

## Overview of the file structure

Root Directory (`whatsapp-int-repo`)

- **.env**: Configuration file containing environment variables.
- **.gitignore**: Lists files and directories to exclude from Git tracking.
- **docker-compose.yml**: Defines services, networks, and volumes for Docker containers.
- **nginx**: Nginx configuration file for managing server routing.
- **VERSION**: A file indicating the version of the application.

### agent Folder

- **app.py**: Main application file for the Agent service.
- **Dockerfile**: Instructions for building the Docker image for the Agent service.
- **requirements.txt**: Lists Python dependencies for the Agent service.

#### Subfolders:

- **services**:
  - **agent\_namespace.py**: Handles Socket.IO namespaces specific to agents.
  - **socketio\_events.py**: Manages Socket.IO events for the Agent service.
- **static**:
  - **styles.css**: Contains styling for the Agent front-end.
  - **js**: JavaScript files for front-end functionality.
- **templates**:
  - **index.html**: Main HTML template for the Agent front-end.
- **utils**:
  - **chat\_room\_helper.py**: Provides helper functions for chat room management.
  - **db\_helper.py**: Contains functions for interacting with the database.
  - **dialog\_360.py**: Integrates Dialog 360 API functionality.
  - **init\_foundations\_helper.py**: Initializes configurations for different foundations.
  - **jwt\_helper.py**: Handles JWT authentication functions.
  - **logging\_config.py**: Configures logging for the Agent service.

### microservice Folder

- **app.py**: Main application file for the Microservice.
- **celery\_app.py**: Celery configuration and task definitions for asynchronous processing.
- **Dockerfile**: Docker image setup for the Microservice.
- **requirements.txt**: Python dependencies for the Microservice.

#### Subfolders:

- **utils**:
  - **chat\_rooms\_helper.py**: Helper functions for managing chat rooms within the microservice.
  - **manage\_foundation\_session.py**: Manages session-based configuration for different foundations.
  - **send\_campaigns\_helpers.py**: Functions to aid in managing campaign sending.

### webhook\_fa Folder

- **app.py**: Main application file for handling webhooks.
- **Dockerfile**: Docker setup for the Webhook service.

- **requirements.txt**: Dependencies for the Webhook service.

#### Subfolders:

- **init\_data**:
  - **languages.json**: Defines language configurations for initialization.
  - **users.json**: Contains user data for initialization.
- **init\_scripts**:
  - **init\_mongo.py**: Initializes MongoDB collections and data.
  - **init\_webhook\_urls.py**: Sets up URLs for webhook endpoints.
- **utils**:
  - **auto\_reply\_helper.py**: Assists in setting up automatic replies.
  - **dialog\_360.py**: Dialog 360 integration specific to the webhook.
  - **generate\_long\_lived\_token.py**: Script for generating long-lived tokens.
  - **helper\_functions.py**: Utility functions for various webhook operations.
  - **init\_webhook\_helper.py**: Sets up initial configurations for the webhook.
- **utils\_init**:
  - **utils\_init.py**: Initialization helper functions for the webhook setup.

#### website Folder

- **Dockerfile**: Docker setup for the Website service.
- **requirements.txt**: Dependencies for the Website service.
- **run.py**: Entry point for running the Website service.

#### Subfolders:

- **app**:
  - **config.py**: Configuration file for the Website.
  - **init.py**: Initializes the Website service.
  - **static**: Contains static assets, such as CSS and JavaScript.
  - **templates**: HTML templates for rendering web pages.
  - **utils**: Utility functions specific to the Website.
  - **views**: Manages routes and views for the Website.
- **sample\_member\_data**:
  - **sample-data.csv**: Example data file for testing or initial setup.

## Now let's explain a little bit more the docker compose file Services

### 1. website

- **build**:
  - **context**: Specifies the build directory for the website service.
  - **dockerfile**: Specifies the Dockerfile to use, though optional if named `Dockerfile`.
- **container\_name**: `website_container`: Names the container for easy reference.
- **restart**: `unless-stopped`: Ensures the container restarts automatically unless explicitly stopped.
- **ports**:
  - `1313:1313`: Exposes the website's Flask app on port 1313, mapping it to the same port on the host.
- **env\_file**: `.env`: Loads environment variables for configuration.
- **volumes**:
  - `./website:/app`: Mounts the local `website` directory to `/app` in the container, allowing file changes to be directly reflected in the container.
- **depends\_on**:
  - Specifies dependencies, ensuring `mongodb` and `webhook_fa` start before the `website` service.
- **networks**: Connects the service to the `network_internal` network.

## 2. webhook\_fa

- **build:**
  - **context:** ./webhook\_fa: Specifies the build directory.
  - **dockerfile:** Defines the Dockerfile to use for building the image.
- **container\_name:** webhook\_fa\_container.
- **restart:** unless-stopped.
- **ports:**
  - 5000:5000: Exposes the FastAPI app on port 5000.
- **env\_file:** .env.
- **volumes:**
  - ./webhook\_fa:/app: Binds the local webhook\_fa directory to /app in the container.
- **depends\_on:** Depends on mongodb and agent services to ensure they are started first.
- **networks:** network\_internal.

## 3. agent

- **build:**
  - **context:** ./agent: Defines the build directory for the agent service.
  - **dockerfile:** Specifies Dockerfile (optional).
- **container\_name:** agent.
- **restart:** unless-stopped.
- **ports:**
  - 5001:5001: Exposes the FastAPI app on port 5001.
- **env\_file:** .env.
- **volumes:**
  - ./agent:/app: Mounts the agent directory to /app in the container.
- **depends\_on:** Ensures mongodb starts before this service.
- **networks:** network\_internal.

## 4. microservice

- **build:**
  - **context:** ./microservice: Defines the build directory for the microservice.
- **container\_name:** microservice.
- **restart:** unless-stopped.
- **ports:**
  - 1616:1616: Maps port 1616 of the container to the same port on the host.
- **volumes:**
  - ./microservice:/app: Maps the microservice directory to /app.
- **env\_file:** .env.
- **command:** Starts the FastAPI application with Uvicorn.
- **networks:** Connects to network\_internal.

## 5. celery\_worker

- **build:**
  - **context:** ./microservice: Uses the same directory for Celery and FastAPI.
- **container\_name:** celery\_worker.
- **restart:** unless-stopped.
- **volumes:**
  - ./microservice:/app.
- **depends\_on:** Depends on rabbitmq and mongodb.
- **env\_file:** .env.

- **command:** Starts the Celery worker with logging enabled.
- **networks:** `network_internal`.

## 6. rabbitmq

- **image:** `rabbitmq:3-management`: Uses RabbitMQ with management capabilities.
- **container\_name:** `rabbitmq`.
- **ports:**
  - `15672:15672`: Maps the management UI to port 15672 on the host.
  - `5672:5672`: Exposes the broker port on 5672.
- **restart:** `unless-stopped`.
- **env\_file:** `.env`.
- **networks:** `network_internal`.

## 7. mongodb

- **image:** `mongodb/mongodb-community-server:latest`: Uses the latest MongoDB community server.
- **container\_name:** `mongodb_container`.
- **restart:** `unless-stopped`.
- **ports:**
  - `27017:27017`: Exposes MongoDB on port 27017.
- **volumes:**
  - `mongo_data:/data/db:z`: Mounts a named volume for data persistence.
- **env\_file:** `.env`.
- **environment:** Sets MongoDB's root username and password using environment variables from `.env`.
- **networks:** `network_internal`

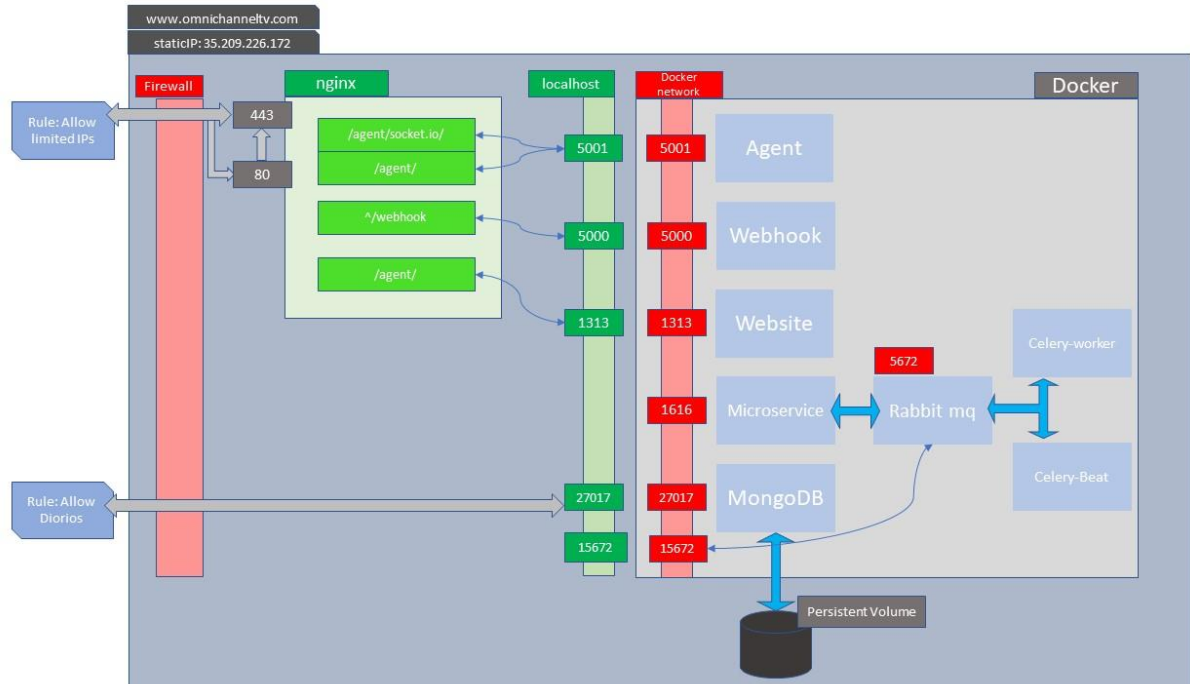
## Volumes

- **mongo\_data:** A named volume to store MongoDB data persistently, ensuring it remains intact if the container restarts or is recreated.

## Networks

- **network\_internal:** A bridge network allowing inter-container communication, isolating the services and securing network traffic within the Docker network.

The following is a visualization of all the containers



## Nginx Configuration Overview

The Nginx configuration is set up with two `server` blocks:

1. **HTTP to HTTPS Redirection:**
  - The first `server` block listens on port 80 for HTTP requests.
  - All HTTP traffic is redirected to HTTPS by returning a 301 status with the `https://$host$request_uri` URL.
2. **HTTPS Server Block:**
  - The second `server` block listens on port 443 (HTTPS).
  - SSL certificates are specified, enabling secure connections for requests to `omnichanneltv.com` and `www.omnichanneltv.com`.

Within this block, specific URL paths are routed to different services in the Docker environment:

- **/webhook:** Requests starting with `/webhook` are forwarded to the `webhook_fa` container on port 5000.
- **/agent/socket.io/:** This handles Socket.IO requests for the `agent` service namespace on port 5001. The `Upgrade` and `Connection` headers are set to manage WebSocket upgrades.
- **/agent/:** Any other requests starting with `/agent/` are directed to the `agent` container on port 5001.
- **Default Route:** Requests to the root (`/`) are forwarded to the `website` service on port 1313.

### Docker Compose File

The `docker-compose.yml` file defines services for each component and organizes them within a shared network (`network_internal`). Key components are:

1. **Website:**
  - Hosts a Flask app exposed on port 1313.
  - Serves as the main web service, receiving traffic from the root URL (`/`).
2. **Webhook:**
  - Runs a FastAPI application for handling webhook-related requests on port 5000.
  - Receives requests directed to the `/webhook` endpoint.
3. **Agent:**
  - Hosts a FastAPI application on port 5001 with Socket.IO and other endpoints.
  - Serves requests directed to the `/agent` paths, including `/agent/socket.io/` for WebSocket communication.
4. **Microservice and Celery Worker:**
  - These services handle background and asynchronous processing tasks.
  - They don't have direct URL mappings in Nginx, so they likely interact internally with other services.

## 5. RabbitMQ and MongoDB:

- These are infrastructure services, with RabbitMQ managing message queues and MongoDB providing a database.
- They are on the internal network for access by other containers, without direct exposure via Nginx.

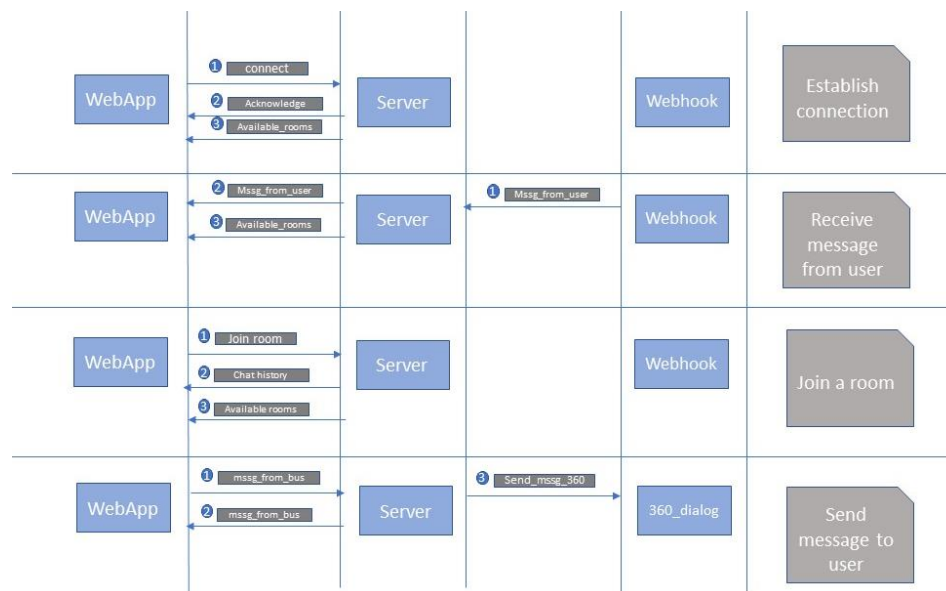
### How URL Routing Works

Requests reaching `omnichanneltv.com` are routed through Nginx based on the URL path:

- `/webhook` requests reach `webhook_fa` at port 5000.
- `/agent` requests reach the `agent` service at port 5001, handling Socket.IO and other endpoints.
- `Root (/)` requests are served by the `website` service running on port 1313.

This setup allows for a single domain to route requests securely and dynamically to various internal services based on the path.

Let's have a small look on how the Socket.IO responsible of handling the agent conversation works



The diagram illustrates a series of connection protocols and events between a WebApp, Server, Webhook, and 360 Dialog API. Here's a breakdown of each section:

- Establish Connection:**
  - **WebApp** sends a `connect` event to the **Server**.
  - **Server** responds by acknowledging the connection and sends an `Available_rooms` event back to **WebApp**.
  - **Purpose:** To establish an initial connection and retrieve a list of available rooms.
- Receive Message from User:**
  - The **Webhook** receives a `Msge_from_user` event and sends this message to the **Server**.
  - The **Server** processes it and sends an `Available_rooms` event to the **WebApp**.
  - **Purpose:** To handle incoming messages from the user and update the WebApp with available rooms accordingly.
- Join a Room:**
  - **WebApp** initiates a `Join room` request to the **Server**.
  - The **Server** responds with the chat history for the room and then sends the list of `Available rooms`.
  - **Purpose:** To allow the WebApp to join a specific chat room, retrieve the chat history, and update available rooms.
- Send Message from Business:**
  - **Server** receives two `msse_from_bus` events from the WebApp.

- **Server** then processes and forwards this as a `Send_msg_360` event to the **360 Dialog**.
- **360 Dialog** API sends the message to the user.
- **Purpose:** To handle outgoing messages from the business side, using the 360 Dialog API to communicate with the user.

Each of these sections represents specific points in the communication protocol to establish and manage chat room connections, handle incoming and outgoing messages, and ensure synchronization between different components of the chat application system.

## Flask App Documentation

### Blueprints

1. **Auth Blueprint** (`auth.py`)
2. **Campaigns Blueprint** (`campaigns.py`)
3. **Templates Blueprint** (`templates.py`)
4. **Upload Data Blueprint** (`uploaddata.py`)

### Auth Blueprint

- **Base URL:** `/auth`

| Method    | Endpoint                                                | Description                                                                                      |
|-----------|---------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| GET       | <code>/</code>                                          | Render the login page.                                                                           |
| POST      | <code>/</code>                                          | Handle user login, verify credentials, set JWT in cookies, and redirect to <code>/index</code> . |
| GET       | <code>/index</code>                                     | Render the index page with campaigns, templates, and foundations.                                |
| GET       | <code>/switch_foundation/&lt;foundation_name&gt;</code> | Switch the current session's foundation and redirect to <code>/index</code> .                    |
| GET, POST | <code>/change_password</code>                           | Change the user's password. Requires old and new passwords.                                      |
| GET, POST | <code>/signup</code>                                    | Create a new user.                                                                               |
| GET       | <code>/logout</code>                                    | Clear session and cookies, then redirect to the login page.                                      |
| GET       | <code>/dynamic_redirect</code>                          | Redirect to a dynamically generated URL based on the current foundation and environment.         |

### Campaigns Blueprint

- **Base URL:** `/campaigns`

| Method | Endpoint                                      | Description                                                              |
|--------|-----------------------------------------------|--------------------------------------------------------------------------|
| GET    | /audience                                     | View a paginated list of audience members, optionally filtered by a tag. |
| DELETE | /audience/<member_id>                         | Remove a specific audience member by ID.                                 |
| DELETE | /audience/tag/<tag>                           | Remove audience members associated with a specific tag.                  |
| GET    | /whatsapp                                     | Display campaigns and templates for the current foundation.              |
| POST   | /send_single_message                          | Send a single message using a selected template.                         |
| GET    | /campaigns                                    | List all campaigns, with formatted date fields.                          |
| GET    | /campaign_details/<campaign_name>             | View detailed information about a specific campaign.                     |
| GET    | /send_campaign                                | Fetch campaigns and templates to render the campaign creation page.      |
| POST   | /send_campaign                                | Schedule and send campaign messages.                                     |
| GET    | /campaign_task_status_polling/<campaign_name> | Poll for the status of a campaign task.                                  |
| GET    | /download/<filename>                          | Download a specific file by filename.                                    |
| GET    | /generate_report_page                         | Render the report generation page with campaign names.                   |
| POST   | /generate_report                              | Generate a report for a specific campaign.                               |
| GET    | /download_csv/<status>                        | Download a CSV file of campaign messages filtered by status.             |
| GET    | /show_all_collections_html                    | View all collections in the database for the current foundation.         |
| POST   | /view_collection_content                      | View the content of a specific collection.                               |

## Templates Blueprint

- **Base URL:** /templates

| Method | Endpoint         | Description                                                 |
|--------|------------------|-------------------------------------------------------------|
| GET    | /templates       | Display all templates for the current foundation.           |
| POST   | /remove_template | Remove a specific template from the database and 360dialog. |

|      |                            |                                                          |
|------|----------------------------|----------------------------------------------------------|
| GET  | /check_submitted_templates | Check the submission status of a specific template.      |
| GET  | /create_template_html      | Render the template creation page.                       |
| POST | /get_template_text         | Retrieve text fields and variable counts for a template. |
| POST | /create_template           | Create and submit a new template to 360dialog.           |

Upload Data Blueprint

- **Base URL:** /upload\_data

| Method       | Endpoint         | Description                                                                 |
|--------------|------------------|-----------------------------------------------------------------------------|
| GET,<br>POST | /upload_file     | Upload a CSV file, preview data, and map columns to the database schema.    |
| GET          | /download_sample | Download a sample CSV file.                                                 |
| POST         | /map_columns     | Map CSV columns to the database schema and validate the data.               |
| POST         | /add_member      | Add a new member manually. Validate the phone number and ensure uniqueness. |

FastAPI Application Documentation

Overview

This FastAPI application integrates with MongoDB, Webhook services, and 360dialog APIs to handle WhatsApp-related tasks, including auto-replies, webhook processing, and managing dynamic collections for multiple foundations.

Project Structure

plaintext  
Copy code

```
.
├── app.py                # Main application entry point
├── Dockerfile            # Docker setup
├── requirements.txt      # Python dependencies
├── init_data/           # Initialization JSON files
│   ├── languages.json   # Language configurations
│   └── users.json       # User data
├── init_scripts/        # Initialization scripts
│   ├── init_mongo.py    # MongoDB initialization
│   └── init_webhook_urls.py # Webhook setup
├── utils/               # Utility scripts
│   └── auto_reply_helper.py
```

```

├── dialog_360.py
├── generate_long_lived_token.py
├── helper_functions.py
├── init_webhook_helper.py
├── utils_init/
│   └── utils_init.py          # Utility initialization logic

```

## API Endpoints

### Webhook Endpoint

*POST /webhook/{foundation\_name}*

Handles incoming webhook events for a specific foundation.

- **Path Parameters:**
  - `foundation_name` (*str*): The name of the foundation for which the webhook is being processed.
- **Headers:**
  - `Authorization` (*Optional*): API key or token for authentication (if applicable).
- **Response:**
  - **200:** {"status": "success"}
  - **400:** {"detail": "Invalid JSON payload"}
  - **404:** {"detail": "Foundation '{foundation\_name}' not found"}
  - **500:** {"detail": "Processing error"}
- **Functionality:**
  - Verifies the `foundation_name`.
  - Processes incoming messages or statuses using foundation-specific handlers.
  - Inserts or updates data in the MongoDB collections.

## Initialization Scripts

MongoDB Initialization (`init_mongo.py`)

Sets up collections and indexes dynamically for each foundation based on the available API keys.

- **Functions:**
  - `create_collections()`:
    - Creates the following collections for each foundation:
      - `whatsapp_data_{foundation_name}`:
        - `campaign`, `campaign_responses`, `language`, `members`, `templates`, `webhook_latest_from_user`, `webhook_latest_to_user`, `webhook_responses`
      - `agent_data_{foundation_name}`:
        - `rooms`
    - Populates initial data from `users.json` and `languages.json`.

## Webhook URL Initialization (`init_webhook_urls.py`)

Configures webhook URLs dynamically for each foundation.

- **Functions:**
  - `init_webhook_urls()`: Initializes webhook URLs for all foundations based on environment variables.
- **Environment Variables:**
  - `FLASK_ENV`: Determines whether the application runs in production or development.
  - `WEBHOOK_BASE_URL` or `NGROK_BASE_URL`: The base URL for webhook endpoints.
- **Webhook URL Format:**
  - `{webhook_base_url}/webhook/{foundation_name}`

## Utilities

### Auto-Reply Handler (`auto_reply_helper.py`)

Automatically replies to users within a 12-hour window.

- **Class:** `AutoReplyHandler`
  - **Methods:**
    - `auto_reply(room_id)`: Sends an auto-reply to the given chat room.
    - `_handle_chat_room(user_phone_number, auto_reply_message, timestamp)`: Manages chat rooms and updates message details.

### Dialog 360 Integration (`dialog_360.py`)

Handles communication with 360dialog for sending messages.

- **Function:** `send_message_to_users_through_360(number, message, api_key_360, logger)`
  - **Parameters:**
    - `number`: Recipient's phone number.
    - `message`: Message text.
    - `api_key_360`: API key for 360dialog.
    - `logger`: Logger for tracking.

### Helper Functions (`helper_functions.py`)

Core logic for processing webhook payloads and managing MongoDB data.

- **Class:** `WhatsAppDataHandler`
  - **Methods:**
    - `process_webhook_payload(payload)`: Processes incoming webhook events.
    - `_handle_user_to_business_messages(value)`: Handles messages from users to the business.

- `_handle_business_to_user_statuses(value)`: Handles statuses for messages sent by the business.
- `_handle_chat_room(sender_phone, wa_mid, message_body, timestamp)`: Updates or creates chat rooms.
- `emit_event(event_name, name_space, data)`: Emits events via Socket.IO.

## Environment Variable Management (`utils_init.py`)

Loads and validates environment variables for MongoDB and API keys.

- **Functions:**
  - `get_mongo_client()`: Establishes a MongoDB connection.
  - `create_collection_if_not_exists(db, collection_name)`: Ensures collections exist.
  - `populate_collection_from_json(collection, json_path, unique_field)`: Inserts initial data from JSON files.
  - `get_360_api_keys()`: Extracts API keys from environment variables.

## Foundation Handlers

### Initialization (`init_webhook_helper.py`)

Dynamically creates handlers for all foundations based on MongoDB configurations.

- **Functions:**
  - `get_foundation_dependencies(foundation_name)`: Retrieves database and API configurations for a foundation.
  - `create_handlers_for_all_foundations()`: Initializes a `WhatsAppDataHandler` for each foundation.
- **Data Flow:**
  1. Retrieves foundation names from the `foundations_db`.
  2. Connects to the agent service via Socket.IO.
  3. Assigns handlers to process webhook events.

## Setup

### Environment Variables

| Variable                   | Description      |
|----------------------------|------------------|
| MONGO_INITDB_ROOT_USERNAME | MongoDB username |
| MONGO_INITDB_ROOT_PASSWORD | MongoDB password |

|                          |                                       |
|--------------------------|---------------------------------------|
| MONGO_HOST               | MongoDB host                          |
| MONGO_PORT               | MongoDB port                          |
| WEBHOOK_BASE_URL         | Webhook base URL in production        |
| NGROK_BASE_URL           | Webhook base URL in development       |
| JWT_SECRET_KEY           | Secret key for JWT generation         |
| {FOUNDATION}_360_API_KEY | API key for each foundation (dynamic) |

## Microservice Documentation

### Overview

This microservice is designed to handle WhatsApp campaign messaging. It uses **FastAPI** for API endpoints, **Celery** for task orchestration, and **MongoDB** for data persistence. The service integrates with 360dialog APIs to send messages and track campaign progress dynamically.

---

### Project Structure

plaintext

Copy code

```
.
├── app.py                # Main FastAPI app entry point
├── celery_app.py         # Celery task definitions and configurations
├── Dockerfile            # Docker setup
├── requirements.txt      # Python dependencies
├── utils/                # Helper scripts
│   ├── chat_rooms_helper.py # Chat room management
│   ├── manage_foundation_session.py # Foundation session management
│   └── send_campaigns_helpers.py # Campaign-related helpers
```

### API Endpoints

#### 1. Start Campaign Task

*POST /start\_campaign\_task/*

- **Description:** Starts a Celery task to send a campaign's messages.
- **Request Body:**

json

Copy code

```
{
  "foundation_name": "foundation_name",
  "campaign_timing": "immediate/scheduled",
  "scheduled_date": "2024-11-25T10:00:00Z",
  "scheduled_date_local": "2024-11-25T13:00:00+03:00",
  "selected_campaign": "campaign_tag",
  "template_json": "JSON string representing template",
  "variables": ["var1", "var2"],
  "campaign_name": "Campaign Title",
  "media_id": "optional_media_id"
}
```

- **Response:**

```
json
Copy code
{
  "task_id": "unique_task_id",
  "message": "Campaign processing started"
}
```

## 2. Get Campaign Task Status

*GET /microservice\_get\_campaign\_status/{task\_id}*

- **Description:** Fetches the status of a campaign task.
- **Path Parameter:**
  - `task_id`: The ID of the Celery task to check the status.
- **Response:**
  - **PENDING:**

```
json
Copy code
{
  "status": "Task is still pending...",
  "task_id": "unique_task_id"
}
```

- **PROGRESS:**

```
json
Copy code
{
  "status": "Task in progress...",
  "total_members": 100,
  "processed": 50,
  "success_count": 40,
  "failed_count": 10,
  "task_id": "unique_task_id"
}
```

- **SUCCESS:**

```
json
```

```
Copy code
{
  "status": "Task completed!",
  "total_members": 100,
  "processed": 100,
  "success_count": 90,
  "failed_count": 10,
  "task_id": "unique_task_id"
}
```

○ **FAILURE:**

```
json
Copy code
{
  "status": "Task failed.",
  "error": "Detailed error message",
  "task_id": "unique_task_id"
}
```

## Task Workflow

Celery Task: `send_message_campaign`

*Description:*

Handles the complete process of sending messages for a campaign.

*Parameters:*

- `foundation_name`: The name of the foundation managing the campaign.
- `campaign_timing`: Whether the campaign is "immediate" or "scheduled."
- `scheduled_date`: ISO 8601 datetime for scheduled campaigns.
- `scheduled_date_local`: Local datetime for scheduled campaigns.
- `selected_campaign`: Campaign tag used to filter members.
- `template_json`: JSON string for the WhatsApp message template.
- `variables`: Variables used in the message template.
- `campaign_name`: The name of the campaign.
- `media_id`: Optional media attachment ID.
- `campaign_id_db`: The MongoDB document ID for tracking campaign progress.

*Workflow:*

### 1. Database Initialization:

- Retrieves `whatsapp_data_db` and `agent_data_db` for the given foundation.
- Initializes `WhatsAppChatCampaignHandler` for managing chat rooms.

### 2. Campaign Scheduling:

- If `campaign_timing` is "scheduled", the task is rescheduled to the provided `scheduled_date`.

### 3. Message Sending:

- Retrieves members from the `members` collection filtered by the `selected_campaign` tag.

- Sends messages using the 360dialog API.
- Updates MongoDB collections with success/failure responses.
- 4. **Progress Updates:**
  - Periodically updates the campaign's progress in the `final_campaign_response` collection.
- 5. **Finalization:**
  - Calculates success/failure percentages and updates the campaign's status to "Finished."

## Utilities

Chat Room Helper (`chat_rooms_helper.py`)

*Class: `WhatsAppChatCampaignHandler`*

- Manages chat rooms in the `rooms` collection.
- Updates or creates new rooms for campaign messages.

*Key Methods:*

- `_handle_chat_room(user_phone_number, wa_mid, campaign_name, timestamp):`
  - Inserts or updates chat rooms with campaign messages.

Foundation Session Manager (`manage_foundation_session.py`)

*Function: `get_dependencies_by_foundation(foundation_name, mongo_url)`*

- Retrieves database and API dependencies for a given foundation.

*Return Values:*

- `whatsapp_data_db`: MongoDB database for WhatsApp data.
- `agent_data_db`: MongoDB database for agent-related data.
- `api_key_360`: API key for 360dialog integration.

Campaign Helpers (`send_campaigns_helpers.py`)

*Functions:*

- `get_member_list(mongo_db, selected_campaign):`
  - Retrieves members tagged with the `selected_campaign`.
- `get_template_json(template_json):`
  - Parses the JSON string for the message template.
- `send_post_request(dialog_360_message_url, api_key_360, template_dict, number, failed_rows, start_time):`
  - Sends a POST request to the 360dialog API and logs the response.
- `manage_campaign_data(...):`
  - Inserts or updates campaign progress in MongoDB.
- `process_response(response, number, response_time, success_rows, failed_rows):`

- Processes API responses and updates success/failure logs.

## Environment Variables

| Variable                   | Description               |
|----------------------------|---------------------------|
| RABBITMQ_DEFAULT_USER      | RabbitMQ username         |
| RABBITMQ_DEFAULT_PASS      | RabbitMQ password         |
| MONGO_INITDB_ROOT_USERNAME | MongoDB username          |
| MONGO_INITDB_ROOT_PASSWORD | MongoDB password          |
| MONGO_HOST                 | MongoDB host              |
| MONGO_PORT                 | MongoDB port              |
| DIALOG_360_MESSAGE_URL     | URL for the 360dialog API |

## Agent Microservice Documentation

### Overview

The Agent microservice is a Flask-SocketIO-based system that manages real-time communication for chat rooms. It supports dynamic namespaces for multiple foundations and includes JWT-based authentication.

### Project Structure

plaintext

Copy code

```

.
├── app.py                # Main Flask app entry point
├── requirements.txt      # Python dependencies
├── services/            # Service-specific code
│   ├── agent_namespace.py # Manages dynamic namespaces for Socket.IO
│   └── socketio_events.py # Handles specific Socket.IO events
├── static/              # Frontend assets
│   ├── styles.css       # Styles for the frontend
│   └── js/              # JavaScript modules
│       ├── domUpdates.js
│       ├── eventListeners.js
│       ├── messageDisplay.js
│       ├── roomManagement.js
│       ├── sendMessage.js
│       ├── socket.js
│       ├── tabSwitch.js
│       └── utils.js
├── templates/          # HTML templates
│   └── index.html       # Main frontend template
└── utils/              # Utility scripts

```

```
├── chat_room_helper.py    # Helper for chat room data
├── db_helper.py           # Database helper functions
├── dialog_360.py          # 360dialog integration helper
├── init_foundations_helper.py # Foundation initialization helper
├── jwt_helper.py          # JWT authentication helper
└── logging_config.py      # Logging configuration
```

---

## Socket.IO Events Documentation

### Namespace Initialization

Namespaces are dynamically created for each foundation using:

```
python
Copy code
/agent/agent_namespace/{foundation_name}
```

Event: `connect`

- **Description:** Triggered when a client connects to the namespace.
- **Namespace:** Dynamic (e.g., `/agent/agent_namespace/{foundation_name}`)
- **Access Control:** JWT token validation is required.
- **Response:**

```
json
Copy code
{
  "message": "Connected successfully"
}
```

Event: `available_rooms`

- **Description:** Sends the list of open and closed rooms to the client.
- **Emitted:**
  - Automatically on `connect`.
  - After room state changes (`new_room`, `message_from_user`, `message_from_business`, etc.).
- **Response Example:**

```
json
Copy code
{
  "openRooms": [
    {"room_id": "room1", "unread_count": 2},
    {"room_id": "room2", "unread_count": 0}
  ],
  "closedRooms": [
    {"room_id": "room3", "unread_count": 0}
  ]
}
```

Event: `new_room`

- **Description:** Creates a new chat room.
- **Data:**

```
json
Copy code
{
  "room_id": "room_name"
}
```

- **Server Action:**
  - Adds the new room to the database.
  - Emits an updated `available_rooms` event.
- **Response:**

```
json
Copy code
{
  "message": "New room created"
}
```

Event: `message_from_user`

- **Description:** Handles messages sent by users to the business.
- **Data:**

```
json
Copy code
{
  "room": "room_id",
  "message": "Hello",
  "timestamp": "2024-11-25T12:00:00Z",
  "sender": "user"
}
```

- **Server Actions:**
  - Stores the message in the `messages` field of the room document.
  - Emits the message to the room.
  - Updates `available_rooms` with the latest state.
- **Response:**
  - **Broadcast:**

```
json
Copy code
{
  "room": "room_id",
  "message": "Hello",
  "sender": "user",
  "timestamp": "2024-11-25T12:00:00Z"
}
```

Event: `message_from_business`

- **Description:** Handles messages sent by the business to users.
- **Data:**

```
json
Copy code
{
  "room": "room_id",
  "message": "Thank you for contacting us",
  "timestamp": "2024-11-25T12:01:00Z",
  "sender": "business"
}
```

- **Server Actions:**
  - Stores the message in the `messages` field of the room document.
  - Sends the message to the 360dialog API for delivery to the user.
  - Emits the message to the room.
  - Updates `available_rooms`.
- **Response:**

```
json
Copy code
{
  "room": "room_id",
  "message": "Thank you for contacting us",
  "timestamp": "2024-11-25T12:01:00Z",
  "sender": "business"
}
```

Event: `join_room`

- **Description:** Allows the client to join a specific room.
- **Data:**

```
json
Copy code
{
  "room": "room_id"
}
```

- **Server Actions:**
  - Adds the client to the room using `join_room`.
  - Marks unread messages as read.
  - Sends the full chat history to the client.
  - Updates `available_rooms`.
- **Response:**

```
json
Copy code
{
  "room": "room_id",
}
```

```
      "messages": [
        {"body": "Hello", "sender": "user", "timestamp": "2024-11-25T12:00:00Z"},
        {"body": "Thank you for contacting us", "sender": "business", "timestamp": "2024-11-25T12:01:00Z"}
      ]
    }
  }
```

Event: `close_conversation`

- **Description:** Closes a conversation.
- **Data:**

```
json
Copy code
{
  "room_id": "room_name"
}
```

- **Server Actions:**
  - Updates the `open_discussion` flag in the database to `false`.
  - Emits an `updated available_rooms` event.
- **Response:**

```
json
Copy code
{
  "status": "closed"
}
```

## Frontend Events

### Available Events

| Event           | Triggered By           | Action                                                    |
|-----------------|------------------------|-----------------------------------------------------------|
| connect         | Connection Established | Displays a success message and initializes the interface. |
| available_rooms | Server                 | Updates the list of open and closed rooms.                |

|                       |                   |                                                             |
|-----------------------|-------------------|-------------------------------------------------------------|
| chat_history          | join_room         | Displays chat history for the selected room.                |
| message_from_user     | Server            | Appends a user message to the chat interface.               |
| message_from_business | Server            | Appends a business message to the chat interface.           |
| close_conversation    | Close Room Button | Closes the selected conversation and updates the room list. |

## Database Schema

Collection: `rooms`

| Field             | Type            | Description                                        |
|-------------------|-----------------|----------------------------------------------------|
| room_id           | string          | Unique identifier for the room.                    |
| created_at        | datetime        | Timestamp of room creation.                        |
| messages          | list of objects | List of messages with sender, body, and timestamp. |
| open_discussion   | boolean         | Flag to indicate if the room is open.              |
| last_message_time | datetime        | Timestamp of the last message in the room.         |

## Environment Variables

| Variable                   | Description                   |
|----------------------------|-------------------------------|
| MONGO_INITDB_ROOT_USERNAME | MongoDB username              |
| MONGO_INITDB_ROOT_PASSWORD | MongoDB password              |
| MONGO_HOST                 | MongoDB host                  |
| MONGO_PORT                 | MongoDB port                  |
| DIALOG_360_MESSAGE_URL     | URL for the 360dialog API     |
| JWT_SECRET_KEY             | Secret key for JWT validation |

## Renewing the SSL Certificate

Step01 enable the firewall rule called allowLetsencrypt

Step02 run `sudo certbot renew`

Step03 disable the firewall rule

